LSIS-AFS Reference Implementation Competition

LunaNet Signal-In-Space - Augmented Forward Signal

Welcome to the LSIS-AFS Reference Implementation Competition! This project challenges teams to build a software reference implementation of NASA's LunaNet Augmented Forward Signal specification for lunar navigation.

Competition Overview

What You're Building

A complete software implementation of the LSIS-AFS digital signal processing chain, including: - **Spreading code generators** (Gold, Weil, Legendre sequences) - **Message encoders/decoders** (BCH, LDPC, CRC) - **Baseband signal generation** (I/Q samples) - **Navigation message parsing** (frame structure, time reconstruction) - **Validation suite** (test vectors, compliance checks)

What You're NOT Building

- RF hardware or transmitters
- Antenna systems
- Receiver front-ends
- Real-time signal acquisition

This is a **pure software competition** focused on digital signal processing algorithms.

Background

LunaNet Overview

LunaNet is NASA's vision for a lunar communications and navigation network - think "GPS for the Moon." The Augmented Forward Signal (AFS) provides Position, Navigation, and Timing (PNT) services to users on and around the Moon.

Technical Context

- **Frequency**: 2492.028 MHz (S-band)
- **Signal Structure**: Dual-channel BPSK (data + pilot)
- **Frame Duration**: 12 seconds
- **Similar to**: GPS L1 C/A, Galileo E1
- **Key Innovation**: Tiered spreading codes for rapid acquisition

Why This Matters

Your implementation will: - Enable lunar navigation system development - Support interoperability testing between providers - Provide reference algorithms for user equipment - Validate the LSIS specification

---

Competition Structure

The Gateway Principle

This competition uses a **gateway-based approach** to break down the complex LSIS-AFS implementation into manageable, incremental milestones. Each gateway represents:

- **A functional capability** that can be independently developed and tested
- **A tangible deliverable** with clear success criteria
- **A building block** that enables subsequent gateways
- **A validation checkpoint** to ensure correctness before proceeding

This approach allows teams to: - Make steady, measurable progress over 6 months - Validate each component before integration - Identify and fix issues early - Demonstrate partial completion even if not all gateways are finished - Work in parallel on independent gateways

Key principle: Each gateway must be fully validated before being considered complete. Quality over speed.

Timeline: 6 Months (March - August 2026)

Competition Start: Monday, 2nd March 2026

Phase 1: Standards Compliance & Architecture (March 2026) - **Deadline**: 31st March 2026 - Study LSIS specification thoroughly - Design system architecture and module interfaces - Plan implementation approach and testing strategy - Set up development environment and tooling - Load and validate reference data (Annex1, Annex3) - Define compliance validation approach

Phase 2: Protocol Implementation (April - July 2026) - **Deadline**: 31st July 2026 - Gateway 1: Spreading Code Generation - Gateway 2: Forward Error Correction (encoding & decoding) - Gateway 3: Navigation Message Framing - Gateway 4: Baseband Signal Generation - Gateway 5: Frame Synchronization & Decoding - Gateway 6: Message Parsing - Gateway 7: Integration & Validation (start)

Phase 3: Validation and Interoperability Demonstration (August 2026) - **Deadline**: 31st August 2026 - Gateway 7: Integration & Validation (complete) - Gateway 8: Documentation & Examples - End-to-end testing and validation - Interoperability demonstration - Final compliance verification - Performance benchmarking

Deliverables

Each gateway produces a **tangible, testable deliverable**:

Gateway	Deliverable	Success Metric
1	Spreading code generators	All 210 PRN codes match references
2	FEC encoding suite	Test vectors validate correctly
3	Frame assembly system	Generate valid 12-second frames
4	I/Q signal files	Binary files ready for testing
5	Frame synchronization	Detect frames in clean signals
6	Message parser	Extract all navigation data
7	End-to-end validation	100% round-trip accuracy + interoperability
8	Documentation	Complete API docs + examples

Getting Started

1. Review the Specification

Read the LSIS-AFS specification:

Key document: - `docs/LSIS-AFS vol A.pdf` - Complete specification with all requirements

Focus on these sections: - Section 1: Introduction and overview - Section 2.3: Signal structure and spreading codes - Section 2.4: Message format and encoding - Appendices C, D, E: Code generation parameters

2. Set Up Your Environment

Requirements: - Ability to perform numerical computations - Signal processing capabilities - Testing framework of your choice

You are free to use: - Any programming language - Any open source libraries or frameworks (proprietary libraries discouraged) - Any development tools

Your implementation must: - Generate correct outputs per specification - Be reproducible - Include clear build/run instructions - Prefer open source dependencies for accessibility and reproducibility

3. Get Reference Data

Download the LSIS specification and annexes: - `docs/LSIS-AFS vol A.pdf` - Main specification - Annex1: LDPC matrices (CSV files) - Annex3: Reference spreading codes (hex files)

These files contain the ground truth for validation.

4. Choose Your Starting Point

Option A: Sequential (Recommended for beginners) - Start with Gateway 1 in Phase 2 (April 2026) - Work through gateways in order - Each gateway builds on previous work

Option B: Parallel (For experienced teams) - Split team across gateways - Gateways 1-4 can be developed in parallel - Gateways 5-6 depend on 1-4 - Gateway 7 integrates everything

Option C: Core-First (For time-constrained teams) - Focus on Gateways 1, 2, 3, 4 (encoding pipeline) - Skip decoding (Gateways 5-6) initially - Validate with test vectors instead of round-trip

---

Project Organization

You are free to organize your code however you prefer. Your submission should include:

Required Components: - Source code implementing the required functionality - Test suite demonstrating correctness - Documentation explaining your approach - Examples showing how to use your implementation - Reference data (LDPC matrices, spreading codes from annexes)

Suggested Organization (not mandatory): - Separate modules for major functional areas - Clear separation between encoding and decoding - Reusable components for common operations - Test files corresponding to implementation files

---

Learning Resources

Understanding the Specification

Start here: 1. Read Section 1 (Introduction) of LSIS-AFS vol A 2. Review Section 2.3.5 (Spreading codes) 3. Study Section 2.4 (Message format) 4. Examine Appendices C, D, E (Code tables)

Sections to focus on (software implementation): - Section 2.3: Signal Structure (spreading codes, modulation) - Section 2.4: Message Format (frame structure, encoding) - Section 2.5: Navigation Message Content - Appendix C: Gold Code Initialization - Appendix D: Weil Primary Code Parameters - Appendix E: Weil Tertiary Code Parameters - Annex 1: LDPC Matrices - Annex 3: Reference Spreading Codes

Sections you can skip (RF hardware, not required): - Section 2.1: RF Characteristics (carrier frequency, power levels, polarization) - Section 2.2: Transmitter Requirements (antenna patterns, EIRP, coverage) - Section 3: User Receiver Performance (sensitivity, acquisition time) - Section 4: Service Performance (accuracy, availability, continuity) - Any sections related to: - Antenna design and patterns - RF power amplifiers

- Frequency stability and phase noise - Transmitter hardware specifications - Link budget calculations - Receiver front-end design

What this means: - You're building the digital baseband signal (I/Q samples) - You don't need to worry about RF carrier generation, amplification, or transmission - Focus on: code generation, encoding, modulation, frame assembly, and the reverse (decoding, parsing) - Your output is I/Q sample files that could be fed to an RF transmitter (but you don't build the transmitter)

Key concepts to understand: - **Gold codes**: Pseudo-random sequences for ranging - **Weil codes**: Long codes derived from Legendre sequences - **Tiered codes**: Hierarchical code structure (primary + secondary + tertiary) - **BCH codes**: Block error correction for critical data - **LDPC codes**: Low-density parity-check codes for bulk data - **BPSK modulation**: Binary phase-shift keying

Similar Systems

If you're familiar with these, you'll recognize patterns: - **GPS L1 C/A**: Similar Gold code structure - **Galileo E1**: Similar tiered code concept - **DVB-S2**: Similar LDPC codes - **802.11n**: Similar LDPC decoder algorithms

Recommended Reading

Digital Signal Processing: - "Understanding Digital Signal Processing" by Richard Lyons - "Software Receiver Design" by C. Fernández-Prades et al.

Error Correction Codes: - "Error Control Coding" by Shu Lin and Daniel Costello - "Modern Coding Theory" by Tom Richardson and Rüdiger Urbanke

GNSS Fundamentals: - "A Software-Defined GPS and Galileo Receiver" by Kai Borre et al. - "Understanding GPS/GNSS" by Elliott Kaplan and Christopher Hegarty

---

Testing Strategy

Test-Driven Development

For each component: 1. **Write tests first** based on specification 2. **Implement** to pass tests 3. **Validate** against reference data 4. **Optimize** while maintaining correctness

Validation Hierarchy

Level 1: Unit Tests - Test individual functions - Use known input/output pairs - Fast execution (< 1 second)

Level 2: Component Tests - Test complete modules - Use test vectors - Moderate execution (< 10 seconds)

Level 3: Integration Tests - Test end-to-end pipelines - Use reference signals - Slower execution (< 1 minute)

Level 4: Compliance Tests - Validate against LSIS specification - Check all requirements - Generate compliance report

Reference Data Sources

From Specification: - Appendix C: Gold code initializations (PRN 1-210) - Appendix D: Weil primary code parameters - Appendix E: Weil tertiary code parameters - Table 11: Interim test code assignments (PRN 1-12)

Generate Your Own: - Encode known data → validate decoder - Generate signals → validate synchronization - Create error patterns → validate FEC

Quick Start Guide

Getting Started

Step 1: Review the specification - Read `docs/LSIS-AFS vol A.pdf` (focus on Sections 2.3-2.5) - Understand the signal structure and message format - Study the spreading code generation algorithms - Understand the deliverables for each gateway

Step 2: Set up your environment - Choose your programming language and tools - Install necessary dependencies for signal processing - Set up version control

Step 3: Start with validation - Load reference data from Annex3 (spreading codes) - Verify you can read and parse the reference codes - This confirms your environment is working

Step 4: Begin implementation - Choose which gateway to start with - Implement and test incrementally - Validate against specification requirements

First Phase Goals

By end of Phase 1 (31st March 2026), you should have: - Thorough understanding of LSIS specification - System architecture designed and documented - Development environment set up - Reference data loaded and accessible - Testing strategy defined - Clear implementation plan for Phase 2 - Compliance validation framework established

---

Tips & Best Practices

Development Tips

Start Simple - Validate one component completely before moving to the next - Test with known values from the specification - Build incrementally and test continuously

Understand Before Implementing - Study the specification thoroughly - Understand the mathematical foundations - Research similar implementations for context (but design your own)

Validate Rigorously - Compare outputs against reference data from annexes - Test edge cases and boundary conditions - Verify compliance with all specification requirements

Common Pitfalls

Avoid: - Skipping validation against reference data - Implementing without understanding the specification - Ignoring edge cases (e.g., PRN 210, frame boundaries) - Hardcoding values that should be derived from specification

Do: - Read the specification carefully - Validate each component independently - Document your design decisions - Test thoroughly at each stage

Getting Help

Specification Questions - Review LSIS-AFS vol A carefully - Check Appendices for details - Look at similar GNSS specifications (GPS ICD, Galileo OS SIS ICD)

Implementation Questions - Study the LSIS specification thoroughly - Review the mathematical definitions in Section 2 - Consult signal processing and coding theory references - Research GNSS receiver design principles

Debugging Tips - Compare intermediate results with specification examples - Validate against reference data from annexes - Test components in isolation before integration - Document your debugging process

---

Support & Communication

Project Resources

Documentation - Specification: `docs/LSIS-AFS vol A.pdf`

Reference Data - LDPC matrices: Available in Annex1 - Spreading codes: Available in Annex3 - Test vectors: Generate from specification

Submission Guidelines

What to Submit 1. Complete source code with clear structure 2. Test suite with results demonstrating correctness 3. Documentation explaining your design and usage 4. Examples demonstrating key functionality 5. Performance report showing compliance with requirements 6. Build/run instructions

Submission Format - Any format that allows evaluation (repository, archive, etc.) - Must include clear instructions for building and running - Must include all dependencies or instructions to obtain them - Should be reproducible on standard hardware

Deadline - Phase 3 completion: 31st August 2026, 23:59 UTC - Late submissions may be accepted with penalty

---

Let's Build!

You now have everything you need to start: - Clear requirements and success criteria - Detailed design and architecture - Step-by-step task breakdown - Reference specification and data - Testing strategy and validation approach

Your next steps: 1. Set up your development environment (Phase 1: March) 2. Read the LSIS-AFS specification thoroughly 3. Design your system architecture 4. Plan your implementation approach 5. Start Gateway 1 implementation in Phase 2 (April) 6. Commit early and often!

Remember: This is a 6-month journey from March to August 2026. Focus on correctness first, then performance. Build incrementally, test continuously, and validate against the specification. Pace yourself across the three phases: Architecture (March), Implementation (April-July), and Validation (August).

Good luck, and happy coding!

License

This reference implementation is provided for educational and competition purposes. The LSIS-AFS specification is maintained by NASA and international partners.

Acknowledgments

- NASA Space Communications and Navigation (SCaN)
- European Space Agency (ESA)

- Japan Aerospace Exploration Agency (JAXA)
- LunaNet Interoperability Specification working group